

PrepME AI — Day 4 Notes (Authentication Foundations)

1. Project Status So Far

Current application architecture:

```
Frontend (React)
  ↓
Axios Request
  ↓
Express Route
  ↓
Controller
  ↓
Mongoose Model
  ↓
MongoDB Database
  ↓
Response
  ↓
Frontend UI
```

This is the foundation of a modern full-stack application.

2. Understanding Signup vs Login

Initially we had a route called:

```
/login
```

But it was actually creating users.

So we learned the difference between:

Signup

Purpose:

Create a new account

Flow:

```
graph TD; A[User enters details] --> B[Check duplicate email]; B --> C[Hash password]; C --> D[Save user];
```

Login

Purpose:

Access an existing account

Flow:

```
graph TD; A[User enters credentials] --> B[Find user]; B --> C[Verify password]; C --> D[Allow access];
```

Signup and Login are different backend operations.

3. Database Read Operations

Previously we only knew:

save()

Example:

```
await newUser.save()
```

Purpose:

Store document in database

This is a WRITE operation.

4. findOne()

Today we learned:

```
const user =  
await User.findOne({ email })
```

Purpose:

Search database
for one matching document

Example:

```
await User.findOne({  
  email: "abc@gmail.com"  
})
```

MongoDB searches the users collection.

5. Return Values of findOne()

User Found

Returns:

```
{  
  email: "abc@gmail.com"  
}
```

User Not Found

Returns:

```
null
```

Example:

```
if (!user)
```

Meaning:

```
No matching user exists
```

6. Duplicate Email Validation

Problem:

Without validation:

```
Same email  
can be inserted  
multiple times
```

Example:

```
abc@gmail.com  
abc@gmail.com  
abc@gmail.com
```

This should not happen.

Solution:

```
const existingUser =  
await User.findOne({ email })  
  
if(existingUser){  
  return res.status(400).json({  
    message: "User already exists"  
  })  
}
```

Purpose:

Prevent duplicate accounts

This is very common production logic.

7. HTTP Status Codes

Backend responses contain status codes.

Examples:

200

Success

Example:

```
res.json({  
  message: "Login successful"  
})
```

400

Bad Request

Example:

```
res.status(400).json({  
  message: "User already exists"  
})
```

500

Server Error

Example:

```
res.status(500).json({  
  message: "Server error"  
})
```

8. Why Frontend Was Not Showing Errors

Backend returned:

400 Bad Request

Axios automatically treats this as an error.

Flow:

```
200 Response  
  ↓  
try block  
  
400 Response  
  ↓  
catch block
```

Previous code:

```
catch(error){  
  console.log(error)  
}
```

Nothing appeared on UI.

Fixed code:

```
catch(error){  
  setMessage(  
    error.response.data.message  
  )  
}
```

Now frontend displays backend errors.

9. Password Security Problem

Initially passwords were stored like:

```
password:  
abc123@
```

This is dangerous.

If database leaks:

```
Everyone's password is exposed.
```

Real applications never store passwords directly.

10. Password Hashing

Today we learned:

```
bcrypt.hash()
```

Installation:

```
npm install bcrypt
```

Example:

```
const hashedPassword =  
await bcrypt.hash(password, 10)
```

Purpose:

Convert password
into secure hash

11. Hashing Example

User enters:

```
abc123@
```

Database stores:

```
$2b$10$9i8...
```

Instead of:

```
abc123@
```

This is industry-standard password storage.

12. Salt Rounds

Example:

```
bcrypt.hash(password, 10)
```

Value:

10

called:

Salt Rounds

Meaning:

How much work bcrypt performs
to generate secure hash

10 is commonly used.

13. Hashing vs Encryption

Encryption

```
Original  
↓  
Encrypted  
↓  
Can be decrypted
```

Hashing

Original
↓
Hashed

Cannot realistically be reversed.

Passwords should be hashed.

14. Login Verification

Today we learned:

```
bcrypt.compare()
```

Example:

```
const isMatch =  
await bcrypt.compare(  
  password,  
  user.password  
)
```

Purpose:

Compare entered password
with stored hash

15. Login Flow

Current login process:

```
User enters email/password  
↓  
Find user by email  
↓
```

```
If user missing
  ↓
User not found
  ↓
Else
  ↓
Compare password
  ↓
If match
  ↓
Login successful
  ↓
Else
  ↓
Invalid password
```

16. Login Controller Structure

Main logic:

```
Find User
  ↓
User Exists?
  ↓
Compare Password
  ↓
Success or Failure
```

This is how most login systems begin.

17. Postman Introduction

Purpose:

```
Test backend APIs
without frontend
```

Very important backend developer tool.

18. Why Postman Is Useful

Without Postman:

Frontend
↓
Backend

Need frontend to test backend.

With Postman:

Postman
↓
Backend

Faster debugging.

19. Postman Request Setup

Method:

POST

URL:

`http://localhost:5000/api/users/login`

Body:

```
{  
  "email": "abc@gmail.com",  
  "password": "abc123@"  
}
```

Body Type:

```
raw
JSON
```

20. API Testing Scenarios

Correct Email + Correct Password

Expected:

```
{
  "message": "Login successful"
}
```

Correct Email + Wrong Password

Expected:

```
{
  "message": "Invalid password"
}
```

User Does Not Exist

Expected:

```
{
  "message": "User not found"
}
```

21. Real Debugging Lesson

Issue faced:

```
User not found
```

even though data existed.

Reason:

Email in Postman:

```
20shwetasingh@gmail.com
```

Email in MongoDB:

```
20singhshweta@gmail.com
```

Different values.

Database query was working correctly.

Lesson:

```
Always verify actual data  
before assuming code is wrong.
```

This is a common backend debugging practice.

22. CRUD Concepts Learned So Far

Create

```
save()
```

Store data.

Read

```
findOne()
```

Retrieve data.

Update and Delete will come later.

23. Current Backend Folder Structure

```
server
|
├── models
|   └── User.js
|
├── routes
|   └── userRoutes.js
|
├── controllers
|   └── userController.js
|
├── config
|   └── db.js
|
├── .env
└── index.js
```

24. Current Authentication Flow

```
Signup
  ↓
Check duplicate email
  ↓
Hash password
  ↓
Store user

Login
```

```
↓  
Find user  
↓  
Compare password  
↓  
Success / Failure
```

25. Most Important Learning Today

Authentication is not:

```
Check email/password
```

Authentication is:

```
Validate user  
Protect credentials  
Store passwords securely  
Handle errors correctly  
Verify login attempts
```

26. Next Learning Goal

Next major topic:

JWT Authentication

Will learn:

- JWT tokens
- token generation
- token verification
- protected routes
- authorization headers
- frontend token storage
- authenticated requests

This is one of the most important backend concepts and a very common interview topic.